LAB MANUAL

Implementation of Word2Vec using Skip-Gram with Negative Sampling (SGNS) in Python

Natural Language Processing Laboratory Experiment

Experiment No.	3
Course	Natural Language Processing Lab
Class / Semester	4th Year 2nd Semester
Student Name	_____
Roll No.	_____
Date of Submission	_____

Prepared for academic laboratory use. This manual is written in a student-friendly, step-by-step format so the code can be implemented and explained easily in class, viva, and record work.

1. Experiment Title

Implementation of a Word2Vec model using the Skip-Gram architecture with Negative Sampling (SGNS) in Python.

2. Aim

To design and implement a simple SGNS-based Word2Vec model from scratch in Python and use it to learn vector representations for words appearing in a small corpus.

3. Learning Objectives

- Understand the idea of distributed word representation and why word embeddings are useful in NLP.
- Explain the Skip-Gram approach where a center word predicts nearby context words.
- Understand the role of Negative Sampling in reducing computation during training.
- Generate training pairs from raw text and train embeddings using gradient-based updates.
- Evaluate the learned vectors using similarity and nearest-neighbor queries.

4. Theory

4.1 Word2Vec Overview

Word2Vec is a neural representation learning technique that maps each word to a dense numerical vector. Words that appear in similar contexts tend to get similar vectors. These vectors help machines capture semantic relationships such as **king ~ queen** and **boy ~ girl**.

4.2 Skip-Gram

In the Skip-Gram model, one word is treated as the center word and the nearby words inside a fixed window are treated as context words. For every center-context pair, the model tries to increase the similarity between the center vector and the true context vector.

4.3 Negative Sampling

Computing probabilities over the complete vocabulary is expensive. SGNS avoids full softmax by updating the model with one positive pair and a few randomly sampled negative words. The training objective therefore does two things:

- pull the center word closer to the true context word, and
- push the center word away from unrelated negative words.

4.4 Model Components

Term	Description
Input embedding matrix	Stores the vector of the center word.
Output embedding matrix	Stores the vector of the context word.
Window size	Controls how many neighboring words are considered as context.
Negative samples	Number of wrong context words used in each update.
Cosine similarity	Measures closeness between two learned word vectors.

5. Software and Hardware Requirements

- Python 3.x
- NumPy library
- A code editor or IDE such as VS Code, PyCharm, or Jupyter
- Basic system capable of running Python programs
- Optional: matplotlib or notebook environment for further analysis

6. Algorithm / Step-by-Step Procedure

1. Read the input text corpus and convert the text into lowercase tokens.
2. Build the vocabulary and assign an index to each unique word.
3. Initialize two embedding matrices: input embeddings and output embeddings.
4. Convert the token sequence into integer indices using the vocabulary map.
5. Generate Skip-Gram training pairs using the chosen context window.
6. For each positive pair, compute the dot product score and apply the sigmoid function.
7. Update the vectors so that the center word moves closer to the true context word.
8. Randomly choose a fixed number of negative samples and update the vectors so that the center word moves away from them.
9. Repeat the training over multiple epochs until the loss decreases and the vectors become meaningful.
10. Use the final input embeddings to compute word similarity and nearest similar words.

Step by step Code

Step 1: Understand what we want to build

We want to build a class called:

SGNSWord2Vec

This class should do 4 main things:

- read text
- create training pairs
- train word vectors
- show similar words

So our final flow will be:

```
model = SGNSWord2Vec(...)
model.fit(text)
model.most_similar("king")
model.similarity("man", "woman")
```

Step 2: Import required libraries

We only need simple libraries:

```
import re
import math
import random
import numpy as np
from collections import Counter
```

Why we need them

- re → for tokenizing text
- math → for sigmoid/log
- random → for shuffling
- numpy → for vectors and matrix operations
- Counter → for word frequency

Step 3: Create the class skeleton

Start with the class and constructor.

```
class SGNSWord2Vec:
    def __init__(self, embedding_dim=20, window_size=2, negative_samples=4,
                 learning_rate=0.03, epochs=200, min_count=1, seed=42):
        self.embedding_dim = embedding_dim
        self.window_size = window_size
        self.negative_samples = negative_samples
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.min_count = min_count
        self.seed = seed
```

What these parameters mean

- embedding_dim → size of each word vector
- window_size → how many nearby words to consider
- negative_samples → how many wrong words to sample
- learning_rate → how fast model learns
- epochs → how many times to train on full data
- min_count → ignore very rare words
- seed → same random results every run

Step 4: Add variables to store vocabulary and embeddings

Inside __init__, add:

```
        self.word_to_idx = {}
        self.idx_to_word = {}
        self.word_counts = Counter()
        self.vocab_size = 0

        self.input_embeddings = None
        self.output_embeddings = None

        np.random.seed(seed)
        random.seed(seed)
```

Meaning

- word_to_idx → maps word to number
- idx_to_word → maps number back to word
- input_embeddings → center word vectors
- output_embeddings → context word vectors

Step 5: Tokenize the text

We now make a function that converts text into words.

```
    def tokenize(self, text):
        text = text.lower()
        tokens = re.findall(r"\b[a-z]+\b", text)
        return tokens
```

Example

If text is:

King is a strong man.

Then tokens become:

["king", "is", "a", "strong", "man"]

Step 6: Build the vocabulary

Now we count words and assign index numbers.

```
    def build_vocab(self, tokens):
        counts = Counter(tokens)

        vocab = []
```

```
for word, count in counts.items():
    if count >= self.min_count:
        vocab.append(word)

self.word_counts = Counter({word: counts[word] for word in vocab})
self.word_to_idx = {word: i for i, word in enumerate(vocab)}
self.idx_to_word = {i: word for word, i in self.word_to_idx.items()}
self.vocab_size = len(vocab)
```

Example

If tokens are:

["king", "is", "a", "strong", "man"]

Then maybe:

```
word_to_idx = {
    "king": 0,
    "is": 1,
    "a": 2,
    "strong": 3,
    "man": 4
}
```

Step 7: Initialize embedding matrices

Each word should get random vectors.

Add this at the end of build_vocab:

```
self.input_embeddings = np.random.uniform(
    -0.5, 0.5, (self.vocab_size, self.embedding_dim)
)
self.output_embeddings = np.random.uniform(
    -0.5, 0.5, (self.vocab_size, self.embedding_dim)
)
```

Shape

If:

- vocab size = 10
- embedding dim = 20

Then:

input_embeddings.shape = (10, 20)
output_embeddings.shape = (10, 20)

Step 8: Convert words into indices

Now we need to convert token list into index list.

```
def tokens_to_indices(self, tokens):
    indices = []
    for word in tokens:
        if word in self.word_to_idx:
            indices.append(self.word_to_idx[word])
    return indices
```

Example

["king", "is", "a"] -> [0, 1, 2]

Step 9: Create training pairs

This is the skip-gram part.

For each center word, pick nearby context words.

```
def generate_training_pairs(self, corpus_indices):
    pairs = []
    n = len(corpus_indices)

    for i, center_idx in enumerate(corpus_indices):
        left = max(0, i - self.window_size)
        right = min(n, i + self.window_size + 1)

        for j in range(left, right):
            if i != j:
                context_idx = corpus_indices[j]
                pairs.append((center_idx, context_idx))

    return pairs
```

Example

For sentence:

king is a strong man

With window_size = 2, if center word is a,
context words may be:

king, is, strong, man

So pairs become:

(a, king)
(a, is)
(a, strong)
(a, man)

Step 10: Sigmoid function

We need sigmoid for prediction.

```
def sigmoid(self, x):
    return 1 / (1 + math.exp(-x))
```

This converts score into value between 0 and 1.

Step 11: Negative sampling

Now choose random wrong words.

```
def sample_negative_indices(self, positive_idx, k):
    negatives = []

    while len(negatives) < k:
        neg_idx = random.randint(0, self.vocab_size - 1)
        if neg_idx != positive_idx:
            negatives.append(neg_idx)

    return negatives
```

Why?

If the correct context is "man", negative samples can be random words like:

- "queen"
- "pretty"
- "young"

These should be pushed away from the center word.

Step 12: Train on one pair

This is the most important step.

Suppose we have one positive pair:

(center = king, context = man)

We:

- bring king and man closer
- push king away from negative words

```
def train_pair(self, center_idx, context_idx):
    loss = 0.0

    v_c = self.input_embeddings[center_idx]
    u_o = self.output_embeddings[context_idx]

    # Positive sample
    score = np.dot(v_c, u_o)
    pred = self.sigmoid(score)
    error = pred - 1

    v_c_old = v_c.copy()
    u_o_old = u_o.copy()

    self.input_embeddings[center_idx] -= self.learning_rate * error * u_o_old
    self.output_embeddings[context_idx] -= self.learning_rate * error * v_c_old

    loss += -math.log(pred + 1e-10)

    # Negative samples
    negative_indices = self.sample_negative_indices(context_idx, self.negative_samples)

    for neg_idx in negative_indices:
        u_k = self.output_embeddings[neg_idx]

        score_neg = np.dot(self.input_embeddings[center_idx], u_k)
        pred_neg = self.sigmoid(score_neg)
        error_neg = pred_neg - 0

        v_c_old = self.input_embeddings[center_idx].copy()
        u_k_old = u_k.copy()

        self.input_embeddings[center_idx] -= self.learning_rate * error_neg * u_k_old
```

```
        self.output_embeddings[neg_idx] -= self.learning_rate * error_neg * v_c_old

        loss += -math.log(1 - pred_neg + 1e-10)

    return loss
```

Step 13: Create the full fit() method

This method will:

- tokenize text
- build vocab
- create pairs
- train for many epochs

```
def fit(self, text):
    tokens = self.tokenize(text)
    self.build_vocab(tokens)
    corpus_indices = self.tokens_to_indices(tokens)
    pairs = self.generate_training_pairs(corpus_indices)

    print("Vocabulary size:", self.vocab_size)
    print("Training pairs:", len(pairs))

    for epoch in range(1, self.epochs + 1):
        random.shuffle(pairs)
        total_loss = 0.0

        for center_idx, context_idx in pairs:
            total_loss += self.train_pair(center_idx, context_idx)

        avg_loss = total_loss / len(pairs)
        if epoch % 20 == 0:
            print(f"Epoch {epoch}/{self.epochs}, Loss: {avg_loss:.4f}")
```

Step 14: Get vector of a word

```
def get_word_vector(self, word):
    idx = self.word_to_idx[word]
    return self.input_embeddings[idx]
```

Step 15: Cosine similarity between two words

```
def similarity(self, word1, word2):
    v1 = self.get_word_vector(word1)
```

```
        v2 = self.get_word_vector(word2)

        num = np.dot(v1, v2)
        den = (np.linalg.norm(v1) * np.linalg.norm(v2)) + 1e-10

        return float(num / den)
```

Step 16: Find most similar words

```
    def most_similar(self, word, top_k=5):
        target_vector = self.get_word_vector(word)
        similarities = []

        for other_word, idx in self.word_to_idx.items():
            if other_word == word:
                continue

            other_vector = self.input_embeddings[idx]

            num = np.dot(target_vector, other_vector)
            den = (np.linalg.norm(target_vector) * np.linalg.norm(other_vector)) + 1e-10
            sim = float(num / den)

            similarities.append((other_word, sim))

        similarities.sort(key=lambda x: x[1], reverse=True)
        return similarities[:top_k]
```

Step 17: Full final code

Now here is the **full easy step-by-step code** in one piece, with your exact target main block style.

```
import re
import math
import random
import numpy as np
from collections import Counter


class SGNSWord2Vec:
    def __init__(
        self,
        embedding_dim=20,
        window_size=2,
        negative_samples=4,
        learning_rate=0.03,
```

```
    epochs=200,
    min_count=1,
    seed=42,
):
    self.embedding_dim = embedding_dim
    self.window_size = window_size
    self.negative_samples = negative_samples
    self.learning_rate = learning_rate
    self.epochs = epochs
    self.min_count = min_count
    self.seed = seed

    self.word_to_idx = {}
    self.idx_to_word = {}
    self.word_counts = Counter()
    self.vocab_size = 0

    self.input_embeddings = None
    self.output_embeddings = None

    np.random.seed(seed)
    random.seed(seed)

# Step 1: Tokenize text
def tokenize(self, text):
    text = text.lower()
    tokens = re.findall(r"\b[a-z]+\b", text)
    return tokens

# Step 2: Build vocabulary
def build_vocab(self, tokens):
    counts = Counter(tokens)

    vocab = []
    for word, count in counts.items():
        if count >= self.min_count:
            vocab.append(word)

    self.word_counts = Counter({word: counts[word] for word in vocab})
    self.word_to_idx = {word: i for i, word in enumerate(vocab)}
    self.idx_to_word = {i: word for word, i in self.word_to_idx.items()}
    self.vocab_size = len(vocab)

    # Step 3: Initialize embeddings
    self.input_embeddings = np.random.uniform(
        -0.5, 0.5, (self.vocab_size, self.embedding_dim)
    )
    self.output_embeddings = np.random.uniform(
        -0.5, 0.5, (self.vocab_size, self.embedding_dim)
```

```
    )

# Step 4: Convert words to indices
def tokens_to_indices(self, tokens):
    indices = []
    for word in tokens:
        if word in self.word_to_idx:
            indices.append(self.word_to_idx[word])
    return indices

# Step 5: Generate skip-gram pairs
def generate_training_pairs(self, corpus_indices):
    pairs = []
    n = len(corpus_indices)

    for i, center_idx in enumerate(corpus_indices):
        left = max(0, i - self.window_size)
        right = min(n, i + self.window_size + 1)

        for j in range(left, right):
            if i != j:
                context_idx = corpus_indices[j]
                pairs.append((center_idx, context_idx))

    return pairs

# Step 6: Sigmoid function
def sigmoid(self, x):
    return 1 / (1 + math.exp(-x))

# Step 7: Sample negative words
def sample_negative_indices(self, positive_idx, k):
    negatives = []

    while len(negatives) < k:
        neg_idx = random.randint(0, self.vocab_size - 1)
        if neg_idx != positive_idx:
            negatives.append(neg_idx)

    return negatives

# Step 8: Train on one pair
def train_pair(self, center_idx, context_idx):
    loss = 0.0

    v_c = self.input_embeddings[center_idx]
    u_o = self.output_embeddings[context_idx]

    # Positive sample
```

```
        score = np.dot(v_c, u_o)
        pred = self.sigmoid(score)
        error = pred - 1

        v_c_old = v_c.copy()
        u_o_old = u_o.copy()

        self.input_embeddings[center_idx] -= self.learning_rate * error * u_o_old
        self.output_embeddings[context_idx] -= self.learning_rate * error * v_c_old

        loss += -math.log(pred + 1e-10)

        # Negative samples
        negative_indices = self.sample_negative_indices(context_idx, self.negative_samples)

        for neg_idx in negative_indices:
            u_k = self.output_embeddings[neg_idx]

            score_neg = np.dot(self.input_embeddings[center_idx], u_k)
            pred_neg = self.sigmoid(score_neg)
            error_neg = pred_neg

            v_c_old = self.input_embeddings[center_idx].copy()
            u_k_old = u_k.copy()

            self.input_embeddings[center_idx] -= self.learning_rate * error_neg * u_k_old
            self.output_embeddings[neg_idx] -= self.learning_rate * error_neg * v_c_old

            loss += -math.log(1 - pred_neg + 1e-10)

        return loss

    # Step 9: Full training
    def fit(self, text):
        tokens = self.tokenize(text)
        self.build_vocab(tokens)
        corpus_indices = self.tokens_to_indices(tokens)
        pairs = self.generate_training_pairs(corpus_indices)

        print("Vocabulary size:", self.vocab_size)
        print("Training pairs:", len(pairs))

        for epoch in range(1, self.epochs + 1):
            random.shuffle(pairs)
            total_loss = 0.0

            for center_idx, context_idx in pairs:
                total_loss += self.train_pair(center_idx, context_idx)
```

```
        avg_loss = total_loss / len(pairs)

        if epoch % 20 == 0:
            print(f"Epoch {epoch}/{self.epochs}, Loss: {avg_loss:.4f}")

    # Step 10: Get vector of a word
    def get_word_vector(self, word):
        idx = self.word_to_idx[word]
        return self.input_embeddings[idx]

    # Step 11: Similarity between two words
    def similarity(self, word1, word2):
        v1 = self.get_word_vector(word1)
        v2 = self.get_word_vector(word2)

        numerator = np.dot(v1, v2)
        denominator = (np.linalg.norm(v1) * np.linalg.norm(v2)) + 1e-10

        return float(numerator / denominator)

    # Step 12: Most similar words
    def most_similar(self, word, top_k=5):
        target_vector = self.get_word_vector(word)
        similarities = []

        for other_word, idx in self.word_to_idx.items():
            if other_word == word:
                continue

            other_vector = self.input_embeddings[idx]

            numerator = np.dot(target_vector, other_vector)
            denominator = (np.linalg.norm(target_vector) * np.linalg.norm(other_vector)) + 1e-10
            sim = float(numerator / denominator)

            similarities.append((other_word, sim))

        similarities.sort(key=lambda x: x[1], reverse=True)
        return similarities[:top_k]


if __name__ == "__main__":
    text = """
    king is a strong man
    queen is a wise woman
    boy is a young man
    girl is a young woman
    prince is a young king
    princess is a young queen
```

```
man is strong
woman is pretty
prince is a boy
princess is a girl
"""

model = SGNSWord2Vec(
    embedding_dim=20,
    window_size=2,
    negative_samples=4,
    learning_rate=0.03,
    epochs=200,
    min_count=1,
    seed=42,
)

model.fit(text)

print("\nMost similar to 'king':")
print(model.most_similar("king"))

print("\nMost similar to 'queen':")
print(model.most_similar("queen"))

print("\nSimilarity('man', 'woman'):")
print(model.similarity("man", "woman"))
```

Step 18: How you should write it easily

A very easy way to write this in exam/class is to remember the code in 6 small parts:

Part 1

Imports + class + constructor

Part 2

tokenize() and build_vocab()

Part 3

tokens_to_indices() and generate_training_pairs()

Part 4

sigmoid() and sample_negative_indices()

Part 5

train_pair() and fit()

Part 6

get_word_vector(), similarity(), most_similar()

So instead of memorizing everything together, students can write it block by block.

Step 19: Small note about output

Because training is random, exact similar-word results may vary a little, but with:

seed=42

you should get stable results most of the time.

Step 20: Best way to explain in viva or class

You can say: In Skip-Gram, we take a center word and predict nearby context words.
In SGNS, instead of checking all vocabulary words, we use one correct context word and a few randomly selected wrong words. During training, the model makes correct words closer and wrong words farther in vector space.

7. Python Program (Student-Friendly Version)

The following program follows the standard SGNS flow in an easy-to-read format. It is suitable for lab record writing, demonstration, and viva explanation.

```
import re
import math
import random
import numpy as np
from collections import Counter


class SGNSWord2Vec:
    def __init__(self, embedding_dim=20, window_size=2, negative_samples=4,
                 learning_rate=0.03, epochs=200, min_count=1, seed=42):
```

```
        self.embedding_dim = embedding_dim
        self.window_size = window_size
        self.negative_samples = negative_samples
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.min_count = min_count
        self.seed = seed

        self.word_to_idx = {}
        self.idx_to_word = {}
        self.word_counts = Counter()
        self.vocab_size = 0

        self.input_embeddings = None
        self.output_embeddings = None

        np.random.seed(seed)
        random.seed(seed)

    def tokenize(self, text):
        text = text.lower()
        tokens = re.findall(r"\b[a-z]+\b", text)
        return tokens

    def build_vocab(self, tokens):
        counts = Counter(tokens)
        vocab = []
        for word, count in counts.items():
            if count >= self.min_count:
                vocab.append(word)

        self.word_counts = Counter({word: counts[word] for word in vocab})
        self.word_to_idx = {word: i for i, word in enumerate(vocab)}
        self.idx_to_word = {i: word for word, i in self.word_to_idx.items()}
        self.vocab_size = len(vocab)

        self.input_embeddings = np.random.uniform(-0.5, 0.5,
                                                  (self.vocab_size, self.embedding_dim))
```

```
    self.output_embeddings = np.random.uniform(-0.5, 0.5,
                                               (self.vocab_size, self.embedding_dim))

def tokens_to_indices(self, tokens):
    return [self.word_to_idx[word] for word in tokens if word in self.word_to_idx]

def generate_training_pairs(self, corpus_indices):
    pairs = []
    n = len(corpus_indices)
    for i, center_idx in enumerate(corpus_indices):
        left = max(0, i - self.window_size)
        right = min(n, i + self.window_size + 1)
        for j in range(left, right):
            if i != j:
                context_idx = corpus_indices[j]
                pairs.append((center_idx, context_idx))
    return pairs

def sigmoid(self, x):
    return 1 / (1 + math.exp(-x))

def sample_negative_indices(self, positive_idx, k):
    negatives = []
    while len(negatives) < k:
        neg_idx = random.randint(0, self.vocab_size - 1)
        if neg_idx != positive_idx:
            negatives.append(neg_idx)
    return negatives

def train_pair(self, center_idx, context_idx):
    loss = 0.0
    v_c = self.input_embeddings[center_idx]
    u_o = self.output_embeddings[context_idx]

    score = np.dot(v_c, u_o)
    pred = self.sigmoid(score)
    error = pred - 1
```

```
        v_c_old = v_c.copy()
        u_o_old = u_o.copy()
        self.input_embeddings[center_idx] -= self.learning_rate * error * u_o_old
        self.output_embeddings[context_idx] -= self.learning_rate * error * v_c_old
        loss += -math.log(pred + 1e-10)

        negative_indices = self.sample_negative_indices(context_idx, self.negative_samples)
        for neg_idx in negative_indices:
            u_k = self.output_embeddings[neg_idx]
            score_neg = np.dot(self.input_embeddings[center_idx], u_k)
            pred_neg = self.sigmoid(score_neg)
            error_neg = pred_neg

            v_c_old = self.input_embeddings[center_idx].copy()
            u_k_old = u_k.copy()
            self.input_embeddings[center_idx] -= self.learning_rate * error_neg * u_k_old
            self.output_embeddings[neg_idx] -= self.learning_rate * error_neg * v_c_old
            loss += -math.log(1 - pred_neg + 1e-10)

        return loss

    def fit(self, text):
        tokens = self.tokenize(text)
        self.build_vocab(tokens)
        corpus_indices = self.tokens_to_indices(tokens)
        pairs = self.generate_training_pairs(corpus_indices)

        print("Vocabulary size:", self.vocab_size)
        print("Training pairs:", len(pairs))

        for epoch in range(1, self.epochs + 1):
            random.shuffle(pairs)
            total_loss = 0.0
            for center_idx, context_idx in pairs:
                total_loss += self.train_pair(center_idx, context_idx)
            avg_loss = total_loss / len(pairs)
            if epoch % 20 == 0:
                print(f"Epoch {epoch}/{self.epochs}, Loss: {avg_loss:.4f}")
```

```
    def get_word_vector(self, word):
        idx = self.word_to_idx[word]
        return self.input_embeddings[idx]

    def similarity(self, word1, word2):
        v1 = self.get_word_vector(word1)
        v2 = self.get_word_vector(word2)
        numerator = np.dot(v1, v2)
        denominator = (np.linalg.norm(v1) * np.linalg.norm(v2)) + 1e-10
        return float(numerator / denominator)

    def most_similar(self, word, top_k=5):
        target_vector = self.get_word_vector(word)
        similarities = []
        for other_word, idx in self.word_to_idx.items():
            if other_word == word:
                continue
            other_vector = self.input_embeddings[idx]
            numerator = np.dot(target_vector, other_vector)
            denominator = (np.linalg.norm(target_vector) * np.linalg.norm(other_vector)) + 1e-10
            sim = float(numerator / denominator)
            similarities.append((other_word, sim))
        similarities.sort(key=lambda x: x[1], reverse=True)
        return similarities[:top_k]


if __name__ == "__main__":
    text = """
    king is a strong man
    queen is a wise woman
    boy is a young man
    girl is a young woman
    prince is a young king
    princess is a young queen
    man is strong
    woman is pretty
    prince is a boy
```

```
princess is a girl
"""

model = SGNSWord2Vec(
    embedding_dim=20,
    window_size=2,
    negative_samples=4,
    learning_rate=0.03,
    epochs=200,
    min_count=1,
    seed=42,
)

model.fit(text)

print("\nMost similar to 'king':")
print(model.most_similar("king"))

print("\nMost similar to 'queen':")
print(model.most_similar("queen"))

print("\nSimilarity('man', 'woman'):")
print(model.similarity("man", "woman"))
```

8. Explanation of the Program Flow

- The constructor stores hyperparameters and creates empty structures for vocabulary and embeddings.
- The tokenize() method converts the raw text into lowercase word tokens.
- The build_vocab() method counts the words, applies min_count filtering, and assigns indices to words.
- The generate_training_pairs() method creates center-context pairs using the selected window size.
- The train_pair() method performs one positive update and several negative updates.
- The fit() method repeatedly trains across all pairs and prints the average loss after fixed intervals.
- The most_similar() and similarity() methods use cosine similarity to inspect the learned word vectors.

9. Expected Output / Observation

After training, the program prints the vocabulary size, number of training pairs, intermediate loss values, and similarity-based queries. Because the model starts with random embeddings and learns

from co-occurrence patterns, words used in similar contexts are expected to become close to each other. Typical qualitative observations from the given corpus are shown below.

Query	Expected observation
most_similar('king')	Words such as prince, man, or queen may appear near the top depending on training dynamics.
most_similar('queen')	Words such as princess, woman, or king may appear near the top.
similarity('man', 'woman')	A positive similarity value is expected because both words occur in related contexts.

10. Result

The SGNS-based Word2Vec model was implemented successfully in Python. The model learned vector representations for words from a small corpus, and the similarity queries demonstrated that words occurring in related contexts move closer together in embedding space.

11. Precautions

- Use a fixed random seed when you want reproducible results for demonstration.
- The corpus should be cleaned and tokenized carefully; punctuation and inconsistent casing can affect the vocabulary.
- A very small corpus may not produce perfect semantic structure, so results should be interpreted qualitatively.
- The learning rate and number of epochs should be chosen carefully; very large values may make training unstable.
- Negative samples must not be the same as the true context word.

12. Viva Questions

- What is the purpose of word embeddings?
- What is the difference between CBOW and Skip-Gram?
- Why is Negative Sampling used in Word2Vec?
- What is the role of the window size?
- Why are two embedding matrices used in SGNS?
- What does cosine similarity measure?
- Why might the result differ slightly across different runs?

13. Conclusion

This experiment demonstrates the complete workflow for implementing a simple NLP embedding model from scratch. Beyond coding practice, it also builds intuition about how contextual learning produces meaningful vector spaces. The same concepts form the foundation for many modern language representation techniques.